Data Visualization with Python

Quick Reference: Matplotlib and pandas

Based on the class notebook Data_Visualization.ipynb

> **Note: How to use this sheet.** Run the examples in order, like a notebook. Each section loads the data it needs. The class CSV files (`enrollment_data.csv`, `student_IQdata.csv`, `student_data.csv`) must be in the same folder as your notebook. No CSV files? Use the code in the Appendix instead.

CONTENTS

1 Overview: Choosing a Chart
2 Setup and Basic Workflow
3 Line Plot from Lists
4 Line Plot from a pandas DataFrame
5 Customizing Lines and Adding a Grid
6 Scatter Plot
7 Histogram
8 Bar Chart
9 Pie Chart
A Practice Data (No CSV Files Needed)

1. Overview: Choosing a Chart

Choose the chart that matches the question you want to answer. All five charts come from `matplotlib.pyplot`, imported as `plt`.

Chart	Function	Use it to show	Class example
Line plot	`plt.plot(x, y)`	How a value changes over time (a trend)	Study hours per day; enrollment per year
Scatter plot	`plt.scatter(x, y)`	Whether two number columns are related (correlation)	Study hours vs IQ score
Histogram	`plt.hist(data)`	How number values are spread out (frequency)	Distribution of IQ scores
Bar chart	`plt.bar(names, counts)`	A comparison of counts between categories	Students per gender, status and location
Pie chart	`plt.pie(counts)`	Each category's share of the whole (%)	Course completion status

> **Note: Histogram vs bar chart.** A histogram groups numbers into ranges (bins) and counts the values in each range. A bar chart counts categories such as Male/Female or Dhaka/Sylhet.

2. Setup and Basic Workflow

2.1 Importing Libraries

Import these three libraries at the top of the notebook. The short names (aliases) save typing.

EXAMPLE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Library	Alias	Used for
NumPy	`np`	Numbers and arrays. Imported in class; used here only for `np.nan` in Appendix A.
pandas	`pd`	Reading and handling tables of data (DataFrames).
matplotlib.pyplot	`plt`	Drawing charts.

Common mistake: Import the plotting module as `import matplotlib.pyplot as plt`. Writing `import matplotlib as plt` does not give you `plt.plot()`.

2.2 Loading Data with pandas

A DataFrame is a table with rows and columns. Load one from a CSV file or build one from a dictionary, then select a column by putting its name in square brackets.

SYNTAX

```
df = pd.read_csv('filename.csv')    # read a CSV file into a DataFrame
df = pd.DataFrame(dictionary)       # build a DataFrame from a dictionary
df['Column Name']                   # select one column
df.head()                           # show only the first 5 rows
```

These are the four tables used in this sheet:

Variable	Source	Columns	Used in
`df`	`enrollment_data.csv`	Year, Programming, Digital Marketing	Sections 4, 5
`students`	`student_IQdata.csv`	Shoe_Size, Study_Hour, Chilling_Hours, IQ_Score	Sections 6, 7
`df`	A dictionary in the code	Student, Gender, StudyHours, Attendance, Grade	Sections 8.1, 8.2
`df`	`student_data.csv`	StudentID, FullName, Data Structure Marks, Algorithm Marks, Python Marks, CompletionStatus, EnrollmentDate, Instructor, Location	Sections 8.3, 9

Common mistake: `df` is reused for three different tables. Running a later `df = ...` line replaces the earlier table, so reload the data when you jump between sections. Column names must match exactly, including capital letters and spaces: `df['Digital Marketing']` works, but `df['digital marketing']` gives a `KeyError`.

Note: Typing just `df` in Jupyter shows the whole table. `df.head()` shows only the first 5 rows, which is handy for long tables.

2.3 Displaying a Chart with plt.show()

Every chart follows the same four steps:

1. Prepare the data (lists or DataFrame columns).
2. Draw the chart (`plt.plot`, `plt.scatter`, `plt.hist`, `plt.bar` or `plt.pie`).
3. Add labels and a title (`plt.xlabel`, `plt.ylabel`, `plt.title`).
4. Display the chart with `plt.show()`.

SYNTAX

```
plt.show()
```

Jupyter Notebook shows charts automatically, but other places (a Python script, VS Code, PyCharm) do not. Ending with `plt.show()` makes the code work everywhere.

> **Note:** In Jupyter, a chart without `plt.show()` may print a line such as `[<matplotlib.lines.Line2D at 0x...>]` above it. It is harmless, and `plt.show()` removes it.

> **Correction:** The class notes used `plt.show()` in only a few cells. It is added to every example here so each one runs in any environment.

3. Line Plot from Lists

A line plot joins points in order. Use it to show how something changes over time, such as study hours over a week.

3.1 Basic Line Plot

SYNTAX

```
plt.plot(x, y)
```

`x` holds the values for the horizontal axis and `y` holds the values for the vertical axis. Here, `hours[0]` belongs to `days[0]`, `hours[1]` to `days[1]`, and so on.

EXAMPLE

```
hours = [2, 3, 4, 1, 5, 7, 3]
days = [1, 2, 3, 4, 5, 6, 7]

# plt.plot(x axis, y axis)
plt.plot(days, hours)
plt.show()
```

> **Common mistake:** Give `x` first, then `y`. Both lists must have the same number of items, or Python raises a `ValueError`.

3.2 Axis Labels and Title

SYNTAX

```
plt.xlabel('text')    # label for the x-axis
plt.ylabel('text')    # label for the y-axis
plt.title('text')     # title at the top of the chart
```

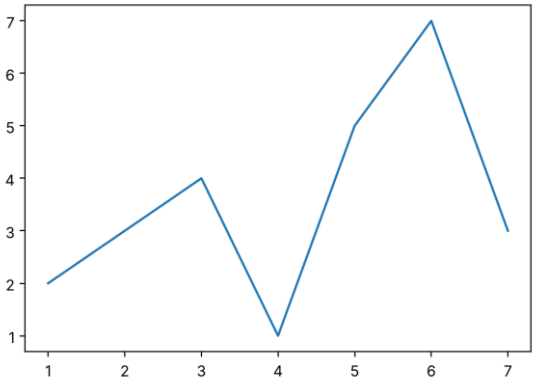
Labels and a title tell the reader what the chart shows. Always add them.

EXAMPLE

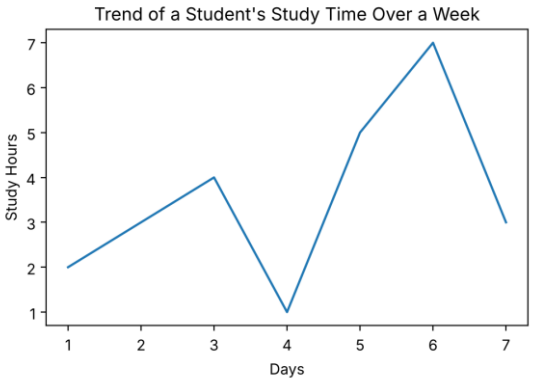
```
# Label for the x-axis
plt.xlabel('Days')
# Label for the y-axis
plt.ylabel('Study Hours')
# Title of the graph
plt.title("Trend of a Student's Study Time Over a Week")

plt.plot(days, hours)
plt.show()
```

OUTPUT



3.1: plt.plot() only



3.2: with labels and title

> **Note:** Put the labels and title in the same cell as `plt.plot()`, before `plt.show()`.

3.3 Plotting Multiple Lines

Call `plt.plot()` once for each line. All lines are drawn on the same chart.

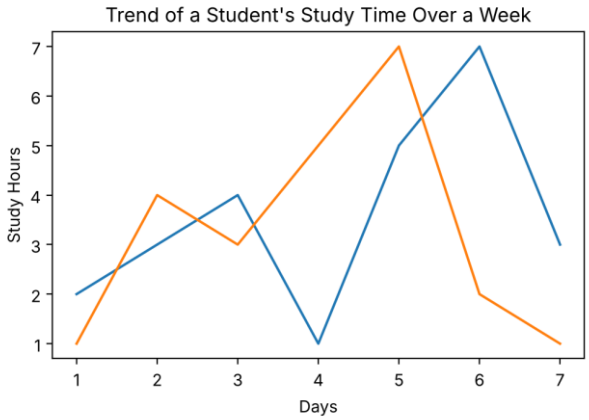
EXAMPLE

```
hours_1 = [2, 3, 4, 1, 5, 7, 3]
hours_2 = [1, 4, 3, 5, 7, 2, 1]
days = [1, 2, 3, 4, 5, 6, 7]

plt.xlabel('Days')
plt.ylabel('Study Hours')
plt.title("Trend of a Student's Study Time Over a Week")

# Two plt.plot() calls = two lines on the same chart
plt.plot(days, hours_1)
plt.plot(days, hours_2)
plt.show()
```

OUTPUT



Common mistake: With two lines and no legend, you cannot tell which line is which. Add `label=` and `plt.legend()` (see Section 4.3).

4. Line Plot from a pandas DataFrame

Real data usually comes from a file. Load it with pandas, then pass DataFrame columns to `plt.plot()`.

4.1 Loading the Data

The table has one row for each year, from 2015 to 2024.

EXAMPLE

```
df = pd.read_csv('enrollment_data.csv')
df
```

OUTPUT

```
   Year  Programming  Digital Marketing
0  2015          589                734
1  2016          862                963
2  2017         1085               1157
3  2018         1150               1404
4  2019         1439               1497
5  2020         1715               1579
6  2021         1785               1948
7  2022         1995               2073
8  2023         2236               2310
9  2024         2442               2338
```

4.2 Plotting DataFrame Columns

SYNTAX

```
plt.plot(df['x_column'], df['y_column'])
```

Select a column with `df['Column Name']` and pass it as `x` or `y`. For two lines, call `plt.plot()` twice.

EXAMPLE 1: ONE COLUMN

```
plt.xlabel('Years')
plt.ylabel('Number of Enrollments')
plt.title('Student Enrollment over 10 Years')

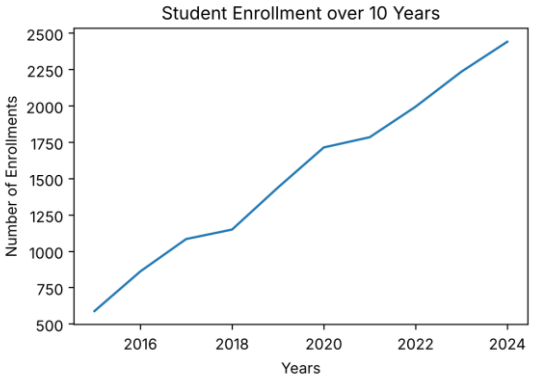
# Pandas columns go straight into plt.plot()
plt.plot(df['Year'], df['Programming'])
plt.show()
```

EXAMPLE 2: TWO COLUMNS

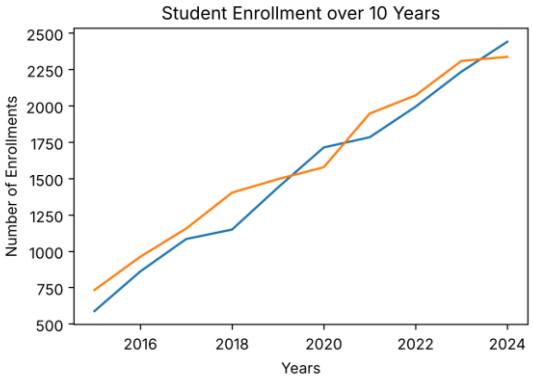
```
plt.xlabel('Years')
plt.ylabel('Number of Enrollments')
plt.title('Student Enrollment over 10 Years')

plt.plot(df['Year'], df['Programming'])
plt.plot(df['Year'], df['Digital Marketing'])
plt.show()
```

OUTPUT



Example 1: Programming only



Example 2: both columns

Correction: The class title said “over 8 years”, but the data covers 2015 to 2024, which is 10 years. The title is corrected here.

4.3 Legend: label and plt.legend()

SYNTAX

```
plt.plot(x, y, label='Line name')   # give the line a name
plt.legend()                        # show the legend
plt.legend(loc='lower right')       # show the legend in a chosen place
```

`label` gives each line a name. `plt.legend()` displays those names in a box. By default, Matplotlib puts the legend where it covers the data least (`loc='best'`).

EXAMPLE 1: DEFAULT LOCATION

```
plt.xlabel('Years')
plt.ylabel('Number of Enrollments')
plt.title('Student Enrollment over 10 Years')

# label = the name of each line (the legend uses it)
plt.plot(df['Year'], df['Programming'], label='Programming')
plt.plot(df['Year'], df['Digital Marketing'], label='Digital Marketing')

# Show the labels (by default the legend goes where there is free space)
plt.legend()
plt.show()
```

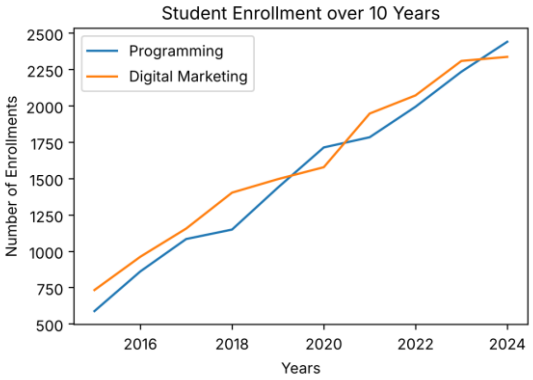
EXAMPLE 2: CHOSEN LOCATION

```
plt.xlabel('Years')
plt.ylabel('Number of Enrollments')
plt.title('Student Enrollment over 10 Years')

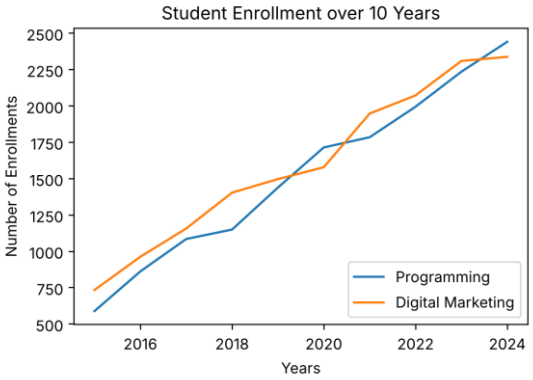
plt.plot(df['Year'], df['Programming'], label='Programming')
plt.plot(df['Year'], df['Digital Marketing'], label='Digital Marketing')

# Choose the legend location yourself
plt.legend(loc='lower right')
plt.show()
```

OUTPUT



plt.legend()



plt.legend(loc='lower right')

Common mistake: `label=` on its own shows nothing. You must also call `plt.legend()`.

Note: `loc` takes a text value such as `'upper left'`, `'upper right'`, `'lower left'` or `'lower right'`.

5. Customizing Lines and Adding a Grid

5.1 Line Style Options

Extra arguments inside `plt.plot()` change how a line looks.

Argument	What it does	Class example
`color`	Line color: a color name or a hex code	`'black'`, `'#11b86f'`
`linewidth`	Thickness of the line	`3`
`linestyle`	Pattern of the line (the default is a solid line)	`'dashed'`
`marker`	Shape drawn on each data point	`'o'` (circle)
`markersize`	Size of the markers	`7`

EXAMPLE

```
plt.xlabel('Years')
plt.ylabel('Number of Enrollments')
plt.title('Student Enrollment over 10 Years')

plt.plot(df['Year'], df['Programming'], label='Programming',
         color='#11b86f', linewidth=3, linestyle='dashed',
         marker='o', markersize=7)
plt.plot(df['Year'], df['Digital Marketing'], label='Digital Marketing',
         color='black', linewidth=3, linestyle='dashed')

plt.legend()
plt.show()
```

Note: A hex code has the form `#RRGGBB` (red, green, blue). `#11b86f` is a green. In this example only the Programming line has markers.

5.2 Adding a Grid

SYNTAX

```
plt.grid()
```

`plt.grid()` draws grid lines so values are easier to read from the chart. Put it before `plt.show()`.

EXAMPLE

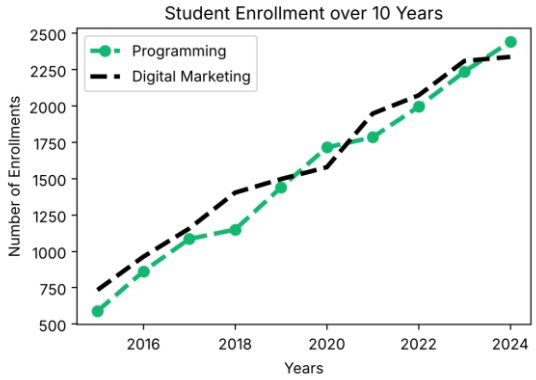
```
plt.xlabel('Years')
plt.ylabel('Number of Enrollments')
plt.title('Student Enrollment over 10 Years')

plt.plot(df['Year'], df['Programming'], label='Programming',
         color='#11b86f', linewidth=3, linestyle='dashed',
         marker='o', markersize=7)
plt.plot(df['Year'], df['Digital Marketing'], label='Digital Marketing',
         color='black', linewidth=3, linestyle='dashed')

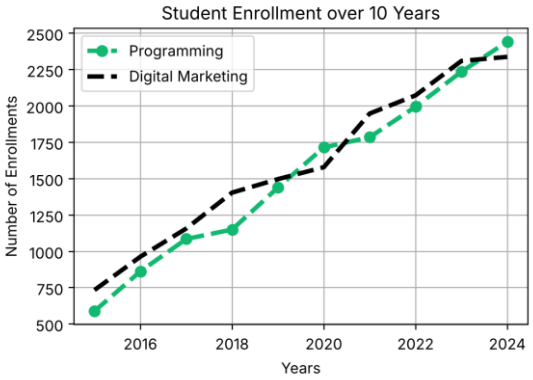
plt.legend()

# Add grid lines to make the values easier to read
plt.grid()
plt.show()
```

OUTPUT



5.1: line styles



5.2: with plt.grid()

6. Scatter Plot

A scatter plot draws one dot for each row of data. Use it to see whether two number columns are related.

6.1 Loading the Data and Reading Correlation

EXAMPLE

```
students = pd.read_csv('student_IQdata.csv')
students.head()
```

OUTPUT (FIRST 5 OF 38 ROWS)

```
   Shoe_Size  Study_Hour  Chilling_Hours  IQ_Score
0       13.5         2.0            17.7        72
1        8.0        18.0             5.0        75
2        6.4         7.2            22.7        78
3        6.9         8.7             6.6       110
4       10.5         8.6             5.8       104
```

Correlation describes the direction of the dots:

Pattern	What you see	Meaning
Positive correlation	Dots go up from left to right	When x increases, y tends to increase
Negative correlation	Dots go down from left to right	When x increases, y tends to decrease
No correlation	Dots are scattered with no direction	x and y are not related

6.2 Positive Correlation: Study Hours vs IQ Score

SYNTAX

```
plt.scatter(x, y)
```

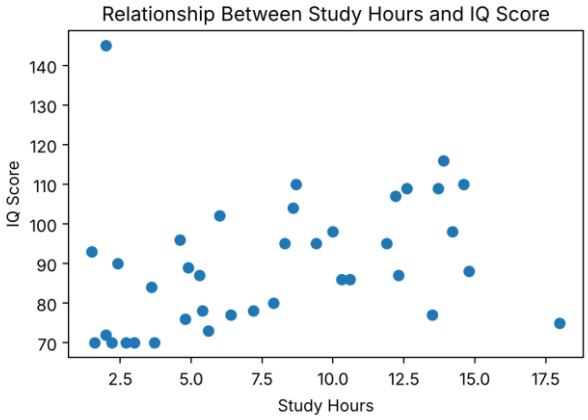
It works like `plt.plot()`: `x` first, then `y`. Do more study hours go with a higher IQ score?

EXAMPLE

```
# Study Hours vs IQ Score: upward trend = positive correlation
plt.xlabel('Study Hours')
plt.ylabel('IQ Score')
plt.title('Relationship Between Study Hours and IQ Score')

# plt.scatter(x axis, y axis)
plt.scatter(students['Study_Hour'], students['IQ_Score'])
plt.show()
```

OUTPUT



Note: The overall direction is upward, so this is a positive correlation, but the trend is weak. One student (2 study hours, IQ 145) is far from the other dots. Real data is rarely a perfect line, so look at the general direction.

6.3 No Correlation: Shoe Size vs IQ Score

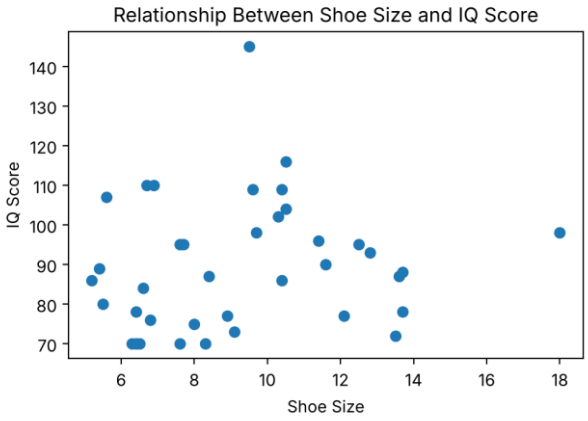
Shoe size and IQ score should be unrelated. The dots show no clear direction.

EXAMPLE

```
# Shoe Size vs IQ Score: no pattern = no correlation
plt.xlabel('Shoe Size')
plt.ylabel('IQ Score')
plt.title('Relationship Between Shoe Size and IQ Score')

plt.scatter(students['Shoe_Size'], students['IQ_Score'])
plt.show()
```

OUTPUT



6.4 Negative Correlation: Chilling Hours vs IQ Score

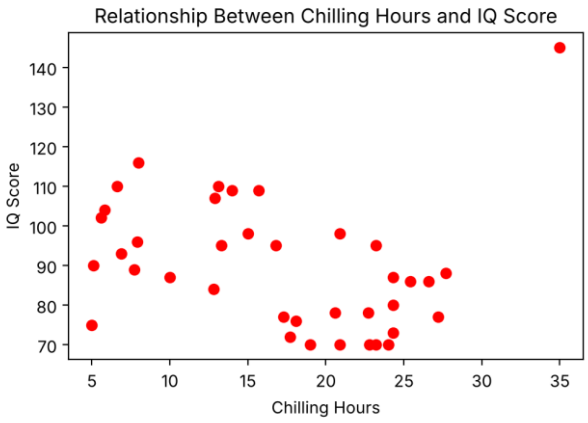
The dots go downward: in this data, students with more chilling hours tend to have lower IQ scores. This example also sets the dot style with `marker='o'` (round dots, which is already the default for scatter plots) and `color='red'`.

EXAMPLE

```
# Chilling Hours vs IQ Score: downward trend = negative correlation
plt.xlabel('Chilling Hours')
plt.ylabel('IQ Score')
plt.title('Relationship Between Chilling Hours and IQ Score')

plt.scatter(students['Chilling_Hours'], students['IQ_Score'],
            marker='o', color='red')
plt.show()
```

OUTPUT



> **Note:** The trend is weak here too: the student with 35 chilling hours has an IQ of 145.

7. Histogram

A histogram shows how number values are spread out. Matplotlib splits the values into ranges called bins and draws one bar for each bin. The bar height is the frequency, the count of values in that bin.

7.1 Basic Histogram

SYNTAX

```
plt.hist(data, bins=10, color='red', edgecolor='black')
```

Argument	What it does	Class example
`bins`	Number of bins, or a list of bin edges (default: 10 bins)	`15`, `[1, 2, 3, 4, 5, 6, 7, 8]`
`color`	Fill color of the bars	`'red'`
`edgecolor`	Border color of the bars, so each bar is easy to see	`'black'`

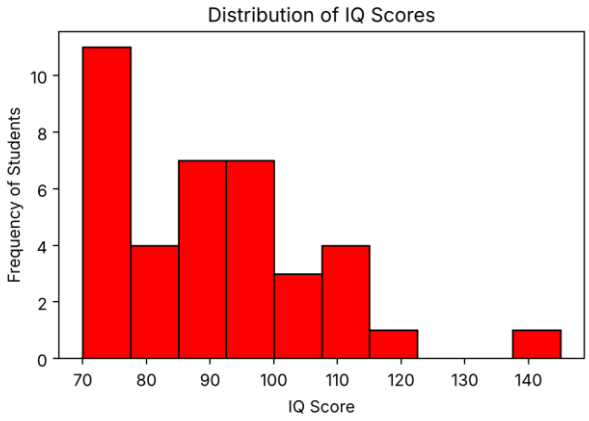
A histogram needs only one list of numbers (no `x` and `y`). This chart shows how many students fall into each IQ score range.

EXAMPLE

```
# Histogram of IQ scores (default: 10 bins)
plt.hist(students['IQ_Score'], color='red', edgecolor='black')

plt.xlabel('IQ Score')
plt.ylabel('Frequency of Students')
plt.title('Distribution of IQ Scores')
plt.show()
```

OUTPUT



7.2 Controlling the Bins

Use `bins=` with a number to choose how many bins to draw, or with a list to choose exactly where each bin starts and ends.

EXAMPLE 1: NUMBER OF BINS

```
# Histogram of study hours with 15 bins
plt.hist(students['Study_Hour'], bins=15, color='red', edgecolor='black')

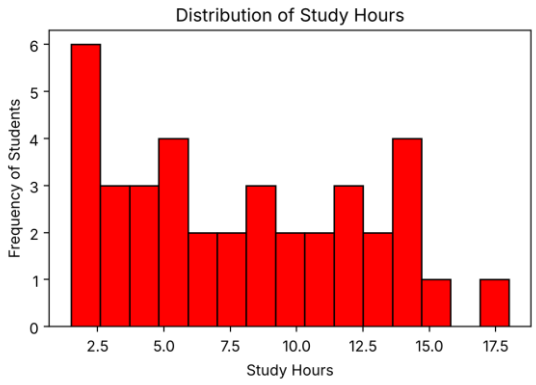
plt.xlabel('Study Hours')
plt.ylabel('Frequency of Students')
plt.title('Distribution of Study Hours')
plt.show()
```

EXAMPLE 2: LIST OF BIN EDGES

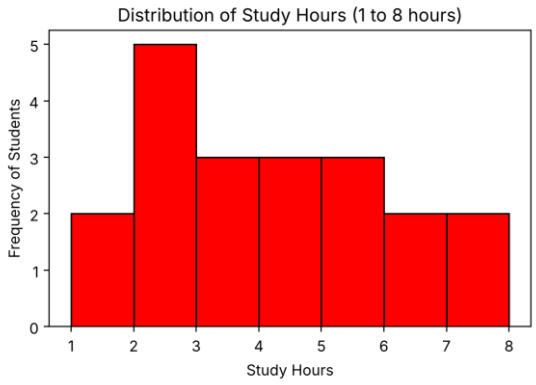
```
# Your own bin edges: 1-2, 2-3, ... 7-8
plt.hist(students['Study_Hour'], bins=[1, 2, 3, 4, 5, 6, 7, 8],
         color='red', edgecolor='black')

plt.xlabel('Study Hours')
plt.ylabel('Frequency of Students')
plt.title('Distribution of Study Hours (1 to 8 hours)')
plt.show()
```

OUTPUT



bins=15



bins=[1, 2, 3, 4, 5, 6, 7, 8]

> **Common mistake:** With a list of edges, values outside the list are not drawn. The 1 to 8 list leaves out every student who studies more than 8 hours, so Example 2 shows only 20 of the 38 students. Make the first and last edges cover your whole data range to include everyone.

> **Note:** Each bin includes its left edge but not its right edge. For example, 2.0 goes into the 2 to 3 bin. The last bin includes both edges.

8. Bar Chart

A bar chart compares counts between categories. First count how many rows belong to each category, then draw one bar for each category.

8.1 Preparing the Counts with groupby().size()

SYNTAX

```
df = pd.DataFrame(dictionary)
counts = df.groupby('Column').size()
```

`groupby('Gender')` puts rows with the same gender into one group, and `.size()` counts the rows in each group. The result has two parts: `.index` (the category names) and `.values` (the counts).

EXAMPLE 1: BUILD THE TABLE

```
data = {
    'Student': ['B', 'C', 'D', 'E', 'F', 'G', 'H'],
    'Gender': ['Male', 'Female', 'Male', 'Female', 'Male', 'Female',
               'Male'],
    'StudyHours': [2, 5, 3, 7, 4, 8, 1],
    'Attendance': [60, 88, 70, 95, 80, 98, 55],
    'Grade': ['Fail', 'Pass', 'Fail', 'Pass', 'Pass', 'Pass', 'Fail']
}

df = pd.DataFrame(data)
df
```

OUTPUT

```
  Student  Gender  StudyHours  Attendance Grade
0       B    Male           2          60  Fail
1       C  Female           5          88  Pass
2       D    Male           3          70  Fail
3       E  Female           7          95  Pass
4       F    Male           4          80  Pass
5       G  Female           8          98  Pass
6       H    Male           1          55  Fail
```

EXAMPLE 2: COUNT EACH GENDER

```
# Count the students in each gender
gender_group = df.groupby('Gender').size()

print(gender_group)               # names on the left, counts on the right
print(list(gender_group.index))   # the category names
print(gender_group.values)        # the counts
```

OUTPUT

```
Gender
Female    3
Male      4
dtype: int64
['Female', 'Male']
[3 4]
```

Common mistake: Every list in the dictionary must have the same number of items, or pandas raises a `ValueError`.

Common mistake: In Jupyter, only the last line of a cell is displayed automatically. If you type `gender_group.index` and then `gender_group.values` on separate lines, you see only the values. Use `print()` to show each one.

Note: Groups are sorted alphabetically (Female, then Male). The colors you list for the bars follow the same order.

8.2 Drawing the Bar Chart

SYNTAX

```
plt.bar(names, heights, color=..., edgecolor=...)
```

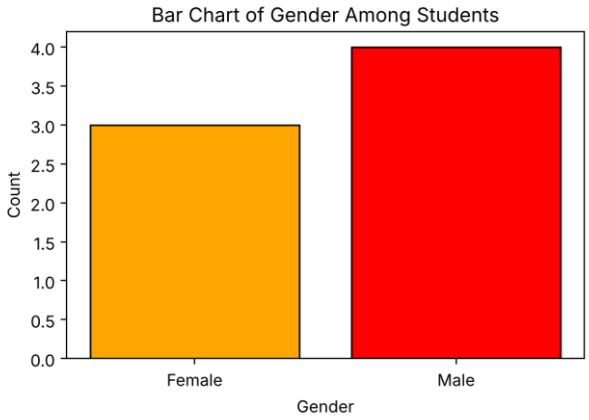
`plt.bar(x, height)` draws one bar for each category. `color` can be one color, or a list with one color for each bar. `edgecolor` draws a border around each bar.

EXAMPLE

```
plt.bar(gender_group.index, gender_group.values,
        color=['orange', 'red'], edgecolor='black')

plt.xlabel('Gender')
plt.ylabel('Count')
plt.title('Bar Chart of Gender Among Students')
plt.show()
```

OUTPUT



8.3 Bar Chart from a CSV File

The same steps work with real data: load the file, count a column with `groupby().size()`, then plot the counts.

EXAMPLE 1: LOAD THE DATA

```
df = pd.read_csv('student_data.csv')
df.head()
```

OUTPUT (FIRST 5 OF 21 ROWS)

```
  StudentID          FullName  Data Structure Marks  Algorithm Marks  \
0    PH1001       Alif Rahman                  85.0             85.0
1    PH1002     Fatima Akhter                  92.0             92.0
2    PH1003     Imran Hossain                  88.0             88.0
3    PH1004  Jannatul Ferdous                  78.0             78.0
4    PH1005       Kamal Uddin                   NaN              NaN

   Python Marks CompletionStatus EnrollmentDate Instructor    Location
0          88.0        Completed     2024-01-15  Mr. Karim       Dhaka
1           NaN      In Progress     2024-01-20  Ms. Salma  Chattogram
2          85.0        Completed     2024-02-10  Mr. Karim       Dhaka
3          82.0        Completed     2024-02-12  Ms. Salma      Sylhet
4          95.0      In Progress     2024-03-05  Mr. Karim  Chattogram
```

EXAMPLE 2: COMPLETION STATUS

```
# Count students by course completion status
status = df.groupby('CompletionStatus').size()

plt.bar(status.index, status.values,
        color=['green', 'orange', 'red'], edgecolor='black')

plt.xlabel('Completion Status')
plt.ylabel('Count')
plt.title('Bar Chart of Course Completion')
plt.show()
```

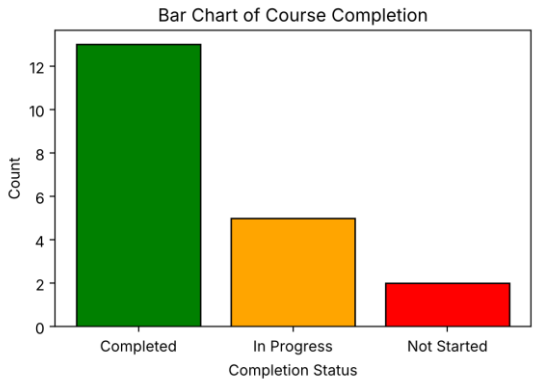
EXAMPLE 3: LOCATION

```
# Count students by location
location_counts = df.groupby('Location').size()

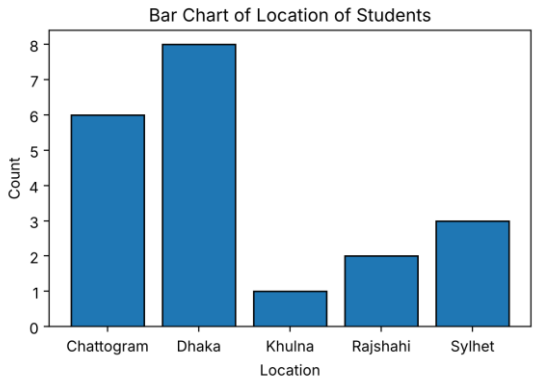
plt.bar(location_counts.index, location_counts.values, edgecolor='black')

plt.xlabel('Location')
plt.ylabel('Count')
plt.title('Bar Chart of Location of Students')
plt.show()
```

OUTPUT



Example 2: completion status



Example 3: location

> **Note:** `groupby()` ignores empty (NaN) values. The last row of `student_data.csv` is completely empty, so it is not counted: the status chart shows 13 + 5 + 2 = 20 students, not 21.

> **Note:** The bars are in alphabetical order, so `color=['green', 'orange', 'red']` means Completed = green, In Progress = orange, Not Started = red.

> **Correction:** In the class notes, the variable `status` was reused for the location counts. It is renamed `location_counts` here, so `status` keeps the completion counts.

9. Pie Chart

A pie chart shows each category's share of the whole. It uses the same counts as a bar chart, but you pass only one set of values (no `x` and `y`).

SYNTAX

```
plt.pie(values, labels=..., autopct=..., explode=..., shadow=True)
```

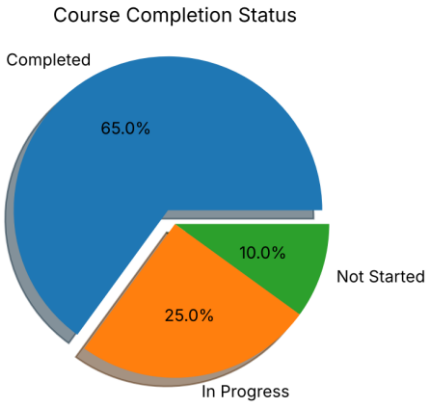
Argument	What it does
`labels`	Names written next to the slices (here, the category names).
`autopct`	Shows each slice's percentage. `'%1.1f%%'` means one decimal place and a % sign.
`explode`	Pulls slices out of the pie. Give one number for each slice: `0.1` moves a slice out by 10% of the radius, `0` leaves it in place.
`shadow`	`True` adds a shadow behind the pie.

EXAMPLE

```
status = df.groupby('CompletionStatus').size()

plt.pie(status, labels=status.index, autopct='%1.1f%%',
        explode=(0.1, 0, 0), shadow=True)
plt.title('Course Completion Status')
plt.show()
```

OUTPUT



Note: Slices follow the order of the data: Completed, In Progress, Not Started. So `explode=(0.1, 0, 0)` pulls out only the Completed slice.

Common mistake: `explode` must have exactly one number for each slice, or Matplotlib raises a `ValueError`. Pass the counts to `plt.pie()`; it calculates the percentages itself.

Appendix A: Practice Data (No CSV Files Needed)

If you do not have the CSV files, run these blocks instead of the matching `pd.read_csv()` lines. Each block builds the same table that was shown in class. Run the imports from Section 2.1 first, because the code uses `pd` and `np.nan` (a missing value).

A.1 enrollment_data.csv (used in Sections 4 and 5)

CODE

```
# Same table as enrollment_data.csv
df = pd.DataFrame({
    'Year': [2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024],
    'Programming': [589, 862, 1085, 1150, 1439, 1715, 1785, 1995, 2236,
                    2442],
    'Digital Marketing': [734, 963, 1157, 1404, 1497, 1579, 1948, 2073,
                          2310, 2338]
})
df
```

A.2 student_IQdata.csv (used in Sections 6 and 7)

CODE

```
# Same table as student_IQdata.csv
students = pd.DataFrame({
    'Shoe_Size': [13.5, 8.0, 6.4, 6.9, 10.5, 6.8, 5.5, 9.6, 9.1, 6.7, 9.7,
                  10.4, 7.7, 12.5, 7.6, 11.4, 5.6, 8.4, 12.1, 6.4, 13.7,
                  13.6, 18.0, 6.3, 11.6, 5.4, 9.5, 10.4, 13.7, 8.3, 6.5,
                  5.2, 7.6, 8.9, 10.5, 12.8, 6.6, 10.3],
    'Study_Hour': [2.0, 18.0, 7.2, 8.7, 8.6, 4.8, 7.9, 12.6, 5.6, 14.6,
                   14.2, 13.7, 11.9, 8.3, 9.4, 4.6, 12.2, 12.3, 6.4, 2.7,
                   5.4, 5.3, 10.0, 2.2, 2.4, 4.9, 2.0, 10.6, 14.8, 1.6,
                   3.0, 10.3, 3.7, 13.5, 13.9, 1.5, 3.6, 6.0],
    'Chilling_Hours': [17.7, 5.0, 22.7, 6.6, 5.8, 18.1, 24.3, 15.7, 24.3,
                       13.1, 20.9, 14.0, 23.2, 13.3, 16.8, 7.9, 12.9,
                       24.3, 17.3, 23.2, 20.6, 10.0, 15.0, 22.8, 5.1, 7.7,
                       35.0, 25.4, 27.7, 19.0, 20.9, 26.6, 24.0, 27.2,
                       8.0, 6.9, 12.8, 5.6],
    'IQ_Score': [72, 75, 78, 110, 104, 76, 80, 109, 73, 110, 98, 109, 95,
                 95, 95, 96, 107, 87, 77, 70, 78, 87, 98, 70, 90, 89, 145,
                 86, 88, 70, 70, 86, 70, 77, 116, 93, 84, 102]
})
students
```

A.3 student_data.csv (used in Sections 8.3 and 9)

CODE

```
# Same table as student_data.csv (note the last empty row)
df = pd.DataFrame({
    'StudentID': ['PH1001', 'PH1002', 'PH1003', 'PH1004', 'PH1005',
                  'PH1006', 'PH1007', 'PH1008', 'PH1009', 'PH1010',
                  'PH1011', 'PH1012', 'PH1013', 'PH1014', 'PH1015',
                  'PH1016', 'PH1017', 'PH1018', 'PH1019', 'PH1020', np.nan],
    'FullName': ['Alif Rahman', 'Fatima Akhter', 'Imran Hossain',
                 'Jannatul Ferdous', 'Kamal Uddin', 'Laila Begum',
                 'Mahmudul Hasan', 'Nadia Islam', 'Omar Faruq',
                 'Priya Sharma', 'Rahim Sheikh', 'Sadia Chowdhury',
                 'Tanvir Ahmed', 'Urmi Akter', 'Wahiduzzaman',
                 'Ziaur Rahman', 'Afsana Mimi', 'Babul Ahmed',
                 'Faria Rahman', 'Nasir Khan', np.nan],
    'Data Structure Marks': [85.0, 92.0, 88.0, 78.0, np.nan, 75.0, 80.0,
```

```
                            81.0, 72.0, 89.0, np.nan, 85.0, 75.0, np.nan,
                            86.0, 94.0, 90.0, 88.0, np.nan, 86.0, np.nan],
    'Algorithm Marks': [85.0, 92.0, 88.0, 78.0, np.nan, 75.0, 80.0, 81.0,
                        72.0, 89.0, np.nan, 85.0, 75.0, np.nan, 86.0,
                        94.0, 90.0, 88.0, np.nan, 86.0, np.nan],
    'Python Marks': [88.0, np.nan, 85.0, 82.0, 95.0, 78.0, np.nan, 85.0,
                     76.0, 88.0, 91.0, 87.0, 79.0, np.nan, 84.0, np.nan,
                     93.0, 85.0, np.nan, 89.0, np.nan],
    'CompletionStatus': ['Completed', 'In Progress', 'Completed',
                         'Completed', 'In Progress', 'Completed',
                         'In Progress', 'Completed', 'Completed',
                         'Completed', 'In Progress', 'Completed',
                         'Completed', 'Not Started', 'Completed',
                         'In Progress', 'Completed', 'Completed',
                         'Not Started', 'Completed', np.nan],
    'EnrollmentDate': ['2024-01-15', '2024-01-20', '2024-02-10',
                       '2024-02-12', '2024-03-05', '2024-03-08',
                       '2024-04-01', '2024-04-22', '2024-05-16',
                       '2024-05-20', '2024-06-11', '2024-06-14',
                       '2024-07-02', '2024-07-09', '2024-08-18',
                       '2024-08-21', '2025-09-01', '2025-09-05',
                       '2025-09-15', '2025-10-02', np.nan],
    'Instructor': ['Mr. Karim', 'Ms. Salma', 'Mr. Karim', 'Ms. Salma',
                   'Mr. Karim', 'Ms. Salma', 'Mr. Karim', 'Ms. Salma',
                   'Mr. David', 'Ms. Salma', 'Mr. Karim', 'Ms. Salma',
                   'Mr. David', 'Ms. Salma', 'Mr. Karim', 'Ms. Salma',
                   'Mr. Karim', 'Ms. Salma', 'Mr. David', 'Ms. Salma',
                   np.nan],
    'Location': ['Dhaka', 'Chattogram', 'Dhaka', 'Sylhet', 'Chattogram',
                 'Rajshahi', 'Dhaka', 'Chattogram', 'Dhaka', 'Sylhet',
                 'Khulna', 'Chattogram', 'Dhaka', 'Rajshahi', 'Dhaka',
                 'Chattogram', 'Dhaka', 'Sylhet', 'Chattogram', 'Dhaka',
                 np.nan]
})
df
```

Note: The A.3 table ends with one row where every value is missing (NaN), exactly as in the class output. `groupby()` ignores that row.